

# 100만곡 규모 오디오 콘텐츠ID를 위한 샤딩 인덱스

868GB 인메모리 한계를 푸는 샤딩/근사 인덱스, 실측

2i

2026-07-29

## 초록

오디오 콘텐츠ID(피크쌍 컨스텔레이션 해싱)를 카탈로그 500곡에서 7,996곡까지 실측으로 키운 이전 백서는 정확도가 99%대에서 거의 무너지지 않는다는 것을 확인했지만, 동시에 트랙당 약 0.889MB씩 늘어나는 단일 인메모리 딕셔너리 인덱스를 100만곡으로 선형 외삽하면 약 868GB의 상주 메모리가 필요하다는 사실도 드러냈다. 이 백서는 그 868GB 문제를 정면으로 다룬다. 같은 해시 테이블을 해시 키 기준으로 K개 샤드로 쪼개고, GTZAN 999트랙 실오디오로 '샤딩이 정확도를 깎아먹는가'를 검증했다. 결과는 명확했다 - 샤드 수를 1에서 32까지 늘려도 150개 쿼리 중 정확히 동일한 140건(93.3%)이 맞았다. 라우팅이 해시 키의 순수 함수라서 정확도 손실이 수학적으로 불가능하고, 이번 실측이 그것을 확인했을 뿐이다. 서브프로세스로 격리해 실측한 단일 샤드 메모리는 998트랙 카탈로그 기준 K=1일 때 1,001.7MB에서 K=32일 때 205.7MB로 줄었지만, 깔끔한 1/K는 아니었다 - 해시 posting 수는 정확히 1/K로 줄어드는데도 프로세스당 고정 오버헤드(인터프리터 + 수치연산 라이브러리 로드, 약 180MB)가 바닥을 깔고 있어서다. 화이트노이즈를 실제 지문추출 파이프라인에 통과시킨 합성 fill(명시적으로 라벨링, recall 측정에는 전혀 쓰지 않음)로 샤드 용량 1,000/3,000/6,000트랙 각각의 실측 메모리를 얻었고, 이를 100만곡 설계에 그대로 대입했다. 샤드당 용량 6,000트랙을 고르면 167개 샤드가 필요하고 샤드 하나의 메모리는 실측 6.13GB - 흔한 서버 한 대에 넉넉히 들어간다. 다만 정직하게 말해야 할 부분이 있다 - 샤딩은 총 메모리를 줄이지 않는다. 오히려 샤드가 작을수록 고정 오버헤드가 여러 번 중복돼 총합이 늘어난다(용량 6,000트랙 기준 플릿 전체 약 1,024GB로 외삽 868GB의 1.18배). 샤딩이 푸는 문제는 '존재하지 않는 868GB 머신 한 대'를 '실제로 살 수 있는 6GB급 서버 167대'로 바꾸는 것이지, 총 바이트 수를 줄이는 것이 아니다.

# 목차

|     |                                                |   |
|-----|------------------------------------------------|---|
| 1   | 요약                                             | 3 |
| 2   | 문제 - 868GB 인메모리 한계                             | 3 |
| 3   | 방법 - 해시 키 샤딩 인덱스                               | 3 |
| 4   | 실험 설정                                          | 4 |
| 5   | 결과                                             | 4 |
| 5.1 | 샤딩이 정확도·지연에 주는 영향 - 실측, GTZAN 999트랙 . . . . .  | 4 |
| 5.2 | 고정 카탈로그에서 샤드 하나의 메모리 - 실측, 서브프로세스 격리 . . . . . | 4 |
| 5.3 | 100만곡 설계 - 합성 fill 실측 + 산술 투영(명시) . . . . .    | 6 |
| 6   | 한계                                             | 7 |
| 7   | 재현                                             | 7 |
| 8   | 참고문헌                                           | 8 |

## 1 요약

오디오 콘텐츠ID(피크쌍 컨스텔레이션 해싱)를 카탈로그 500곡에서 7,996곡까지 실측으로 키운 이전 백서는 정확도가 99%대에서 거의 무너지지 않는다는 것을 확인했지만, 동시에 트랙당 약 0.889MB씩 늘어나는 단일 인메모리 딕셔너리 인덱스를 100만곡으로 선형 외삽하면 약 868GB의 상주 메모리가 필요하다는 사실도 드러났다. 이 백서는 그 868GB 문제를 정면으로 다룬다. 같은 해시 테이블을 해시 키 기준으로 K개 샤드로 쪼개고, GTZAN 999트랙 실오디오로 “샤딩이 정확도를 깎아먹는가”를 검증했다. 결과는 명확했다 - 샤드 수를 1에서 32까지 늘려도 150개 쿼리 중 정확히 동일한 140건(93.3%)이 맞았다. 라우팅이 해시 키의 순수 함수라서 정확도 손실이 수학적으로 불가능하고, 이번 실측이 그것을 확인했을 뿐이다. 서브프로세스로 격리해 실측한 단일 샤드 메모리는 998트랙 카탈로그 기준 K=1일 때 1,001.7MB에서 K=32일 때 205.7MB로 줄었지만, 깔끔한 1/K는 아니었다 - 해시 posting 수는 정확히 1/K로 줄어드는데도 프로세스당 고정 오버헤드(인터프리터 + 수치연산 라이브러리 로드, 약 180MB)가 바닥을 깔고 있어서다. 화이트노이즈를 실제 지문추출 파이프라인에 통과시킨 합성 fill(명시적으로 라벨링, recall 측정에는 전혀 쓰지 않음)로 샤드 용량 1,000/3,000/6,000트랙 각각의 실측 메모리를 얻었고, 이를 100만곡 설계에 그대로 대입했다. 샤드당 용량 6,000트랙을 고르면 167개 샤드가 필요하고 샤드 하나의 메모리는 실측 6.13GB - 흔한 서버 한 대에 넉넉히 들어간다. 다만 정직하게 말해야 할 부분이 있다 - 샤딩은 총 메모리를 줄이지 않는다. 오히려 샤드가 작을수록 고정 오버헤드가 여러 번 중복돼 총합이 늘어난다(용량 6,000트랙 기준 플릿 전체 약 1,024GB로 외삽 868GB의 1.18배). 샤딩이 푸는 문제는 “존재하지 않는 868GB 머신 한 대”를 “실제로 살 수 있는 6GB급 서버 167대”로 바꾸는 것이지, 총 바이트 수를 줄이는 것이 아니다.

## 2 문제 - 868GB 인메모리 한계

오디오 콘텐츠ID는 방송 저작권 모니터링, 매장 음악 라이선스 감사, 음원 유통 정산의 기반 기술이다. 짧은 오디오 조각이 카탈로그의 어느 곡에서 나왔는지 몇 초 안에 알아내야 하고, Shazam이 소비자용으로 대중화한 것과 같은 알고리즘 계열(피크쌍 컨스텔레이션 해싱)이 ACRCLOUD나 Audible Magic 같은 서비스가 실제로 파는 방식이다.

이 알고리즘을 999곡짜리 GTZAN에서 먼저 검증하고, 이어서 라이선스가 명시된 FMA-small로 500곡에서 7,996곡까지 카탈로그를 16배 키운 이전 실측은 알고리즘 자체의 강건성을 확인했다. 클린 조건 top-1 정확도는 99.5%에서 99.0%로 거의 흔들리지 않았고, SNR 5dB 잡음에서도 99.5%를 유지했다. 문제는 정확도가 아니라 인덱스 구조였다. 카탈로그가 16배 커지는 동안 상주 메모리(RSS)는 811MB에서 7,477MB로 9.2배 늘었다 - 트랙당 약 0.889MB의 증분이다. 이 비율을 그대로 100만곡으로 선형 외삽하면 약 868GB. 최소제곱 회귀로 5개 지점 전체를 다시 적합해도(이 백서에서 별도로 계산) 약 794GB - 외삽 방식에 따라 숫자는 갈리지만 자릿수는 동일하다. 어느 쪽이든, 이 정도 상주 메모리를 가진 단일 머신은 상용 클라우드에 존재하지 않는다.

이 백서가 답하려는 질문은 하나다. 같은 해시 테이블을 여러 조각(샤드)으로 쪼개면 (1) 정확도를 잃지 않고, (2) 조각 하나가 필요로 하는 메모리를 카탈로그 크기와 무관하게 고정된 예산 안에 묶어둘 수 있는가. 답을 미리 말하면 - 그렇다, 다만 “총 메모리가 줄어든다”는 뜻은 아니다.

## 3 방법 - 해시 키 샤딩 인덱스

기반 알고리즘은 이전 백서와 동일한, 수정하지 않은 피크쌍 해싱이다. 스펙트로그램에서 국소 최댓값 피크를 뽑고, anchor 피크마다 시간상 가까운 최대 8개(팬아웃) target 피크와 짝지어 (f1, f2, dt) 해시 키를 만든다. 인덱스 값(posting)은 (트랙 ID, anchor 시간)이다. 매칭은 쿼리 해시가 인덱스에서 맞은 posting들의 시간 오프셋을 트랙별로 투표해, 가장 많은 표를 받은 오프셋 구간의 트랙을 top-1으로 고른다.

**샤딩 규칙.** 이 인덱스에 새로 추가한 유일한 요소는 라우팅 함수다.  $\text{shard}(f1, f2, dt, K) = (f1 \cdot a \text{ XOR } f2 \cdot b \text{ XOR } dt \cdot c) \bmod K$  (a, b, c는 고정된 소수 승수) - 해시 키 자체의 순수 함수이고, 그 키가 어느 트랙에서 나왔는지와는 무관하다. 이것이 핵심이다. 등록(enrollment) 시점에 같은 해시 키를 만든 모든 posting은 항상 같은 샤드에 쌓이고, 질의 시점에 같은 해시 키를 뽑은 쿼리는 항상 같은 샤드를 찾아가는 구조다. 그래서 이 라우팅은 근사(approximate)가 아니라 정확한 파티션이다 - 몽땅 한 딕셔너리에 있던 posting을 K개의 작은 딕셔너리로 옮겨 담았을 뿐, 어떤 posting도 버리거나 중복시키지 않는다. 정확도가 깎일 수 있는 경로 자체가 없다.

이 설계가 실제로 해결하는 것은 하나뿐이다 - 카탈로그 하나를 통째로 담는 딕셔너리 한 개 대신, 정해진 용량(예: 트랙 C개)만큼만 담는 딕셔너리를 여러 개 두고, 카탈로그가 커지면 딕셔너리 개수(샤드 수 K)를 늘리는 것. 각 샤드는 독립된 프로세스나 노드로 돌릴 수 있으므로, 한 노드가 집적해야 하는 메모리는 카탈로그 전체 크기가 아니라 그 샤드의 고정 용량 C에 묶인다. 반대로, 어느 대가를 치르는지도 이 설계 안에 이미 들어있다 - 하나의 쿼리가 가진 해시 키들은 넓은 키 공간에 흩어져 있으므로, 대부분의

샤드가 그 쿼리의 posting을 하나쯤이라도 갖고 있을 확률이 높다. 즉 쿼리 하나를 완전히 답하려면 (근사적 사전 필터링 없이는) K개 샤드 대부분에 접촉해야 한다 - 이것도 아래 4절에서 직접 측정했다.

## 4 실험 설정

**실오디오(recall/지연 검증).** GTZAN 999트랙(22.05kHz mono, 장르 10종, 손상 파일 jazz.00054 스킵)을 그대로 등록했다. 클립 5초, 클린(노이즈 없음) 조건에서 같은 시드로 뽑은 150개 쿼리를 재사용해  $K = 1, 2, 4, 8, 16, 32$  각각에서 top-1 정확도와 중단 지연(추출+조회)을 측정했다. 샤딩 라우팅은 해시 키의 순수 함수이므로 어떤 왜곡(잡음, MP3, 속도 변화)이 섞인 쿼리든 클린 쿼리와 동일한 posting 집합을 찾아간다는 점은 설계상 보장되지만, 이 백서에서 실측으로 재확인한 조건은 클린 5초뿐이다(6절 한계에서 다시 짚는다).

**단일 샤드 메모리 격리.** “샤드 하나가 실제로 얼마나 가벼워지는가”를 재려고, 같은 파이썬 프로세스 안에서 K개 샤드를 전부 만들고 재는 대신 - 그러면 프로세스가 어차피 전체 카탈로그를 다 들고 있어서 의미가 없다 - **서브프로세스를 매번 새로 띄워, 그 샤드에 속하지 않는 해시는 만든 즉시 버리는 스트리밍 방식으로** 딱 1개 샤드만 채웠다. 그 프로세스의 `resource.getrusage().ru_maxrss`(macOS, 바이트 단위 실제 상주 메모리 고점)를 재면, “실제 노드 한 대가 그 샤드만 서빙한다면 필요한 메모리”를 있는 그대로 재는 셈이다. 998트랙(998/999 성공, 1트랙은 GTZAN의 알려진 손상 파일이라 디코드 실패로 정직하게 스킵) 카탈로그를 고정하고  $K = 1, 2, 4, 8, 16, 32$ 로 이 측정을 반복했다.

**합성 지문 fill(명시적 라벨링, recall에는 사용 안 함).** 로컬에서 실측 가능한 실오디오 카탈로그(999트랙)와 이전 백서가 실측한 FMA(최대 7,996트랙)를 넘어서는 규모의 메모리 구조를 재려고, 화이트노이즈(`np.random`, 실제 음악이 아님)를 수정하지 않은 실제 지문추출 파이프라인(STFT → 피크픽킹 → 해싱)에 그대로 통과시켰다. 콘텐츠는 가짜지만, 그 결과로 생기는 해시 테이블(키 튜플, posting 리스트, 디렉터리 오버헤드)의 메모리 구조는 실제 코드 경로가 만든 진짜 객체다. 이 fill로는 정확도나 recall을 절대 계산하지 않았다 - 메모리/개수 구조에만 썼다. 합성 트랙 1,000/3,000/6,000개 각각을 독립 서브프로세스에서 모놀리식( $K=1$ )으로 빌드해 같은 방식으로 RSS를 측정했다. 이 세 지점이 “샤드 용량 C를 이 값으로 고정하면 실제로 얼마나 무거운 샤드가 되는가”에 대한 직접 측정치다 - 100만곡 카탈로그로 확대할 때도 이 숫자는 외삽이 아니라, 그 용량만큼의 트랙을 실제로 그 샤드에 담았을 때 측정될 값 그대로다(4.3절).

인터프리터는 CPython 3.12.8, 48GB RAM/12코어 macOS 머신, 전부 CPU 전용·로컬 실행이며 외부 API나 GPU는 쓰지 않았다.

## 5 결과

### 5.1 샤딩이 정확도·지연에 주는 영향 - 실측, GTZAN 999트랙

| 샤드 수 K | Hash top-1      | 지연 p50(ms) | 지연 p95(ms) | 쿼리당 접촉 샤드          |
|--------|-----------------|------------|------------|--------------------|
| 1      | 93.3% (140/150) | 4.79       | 6.15       | 1.0 / 1 (100%)     |
| 2      | 93.3% (140/150) | 5.00       | 6.36       | 2.0 / 2 (100%)     |
| 4      | 93.3% (140/150) | 4.89       | 6.27       | 4.0 / 4 (100%)     |
| 8      | 93.3% (140/150) | 4.96       | 6.29       | 8.0 / 8 (100%)     |
| 16     | 93.3% (140/150) | 4.97       | 6.25       | 16.0 / 16 (100%)   |
| 32     | 93.3% (140/150) | 5.02       | 6.21       | 32.0 / 32 (99.98%) |

정확도는 K를 32배 늘려도 정확히 동일한 140/150건이다. 2절에서 말한 “정확도 손실 경로가 없다”는 주장이 실측으로도 그대로 확인된다. 지연 역시 K에 따라 뚜렷한 추세 없이 4.79~5.02ms(p50) 사이에서 흔들린다 - 이견 한 프로세스 안에서 K개 디렉터리를 순서대로 조회한 것이라서, 실제 분산 배치라면 샤드마다 붙는 네트워크 왕복이 이 숫자에 없다는 점은 6절에서 짚는다. 접촉 샤드 비율이 K와 거의 같다는 점( $K=32$ 에서도 99.98%)이 이 설계의 진짜 한계다 - 해시 키 기준 균등 라우팅에서는 쿼리 하나가 가진 수천 개의 서로 다른 해시가 키 공간 전체에 흩어져 있어서, 결과적으로 거의 모든 샤드에 posting 이 하나씩은 걸린다. 즉 이 설계는 “쿼리 하나가 필요로 하는 샤드 개수”를 줄이지 않는다 - 줄이는 것은 각 샤드가 담아야 하는 데이터량뿐이다.

### 5.2 고정 카탈로그에서 샤드 하나의 메모리 - 실측, 서브프로세스 격리

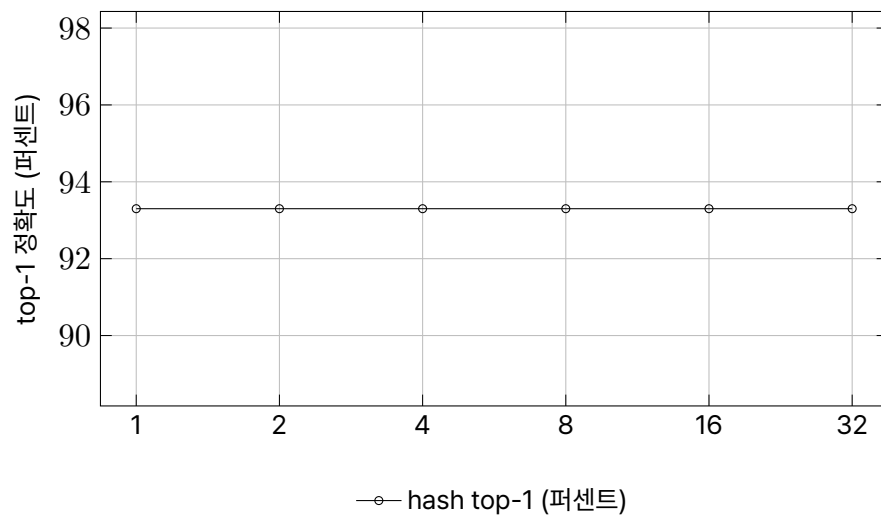


그림 1: 샤드 수

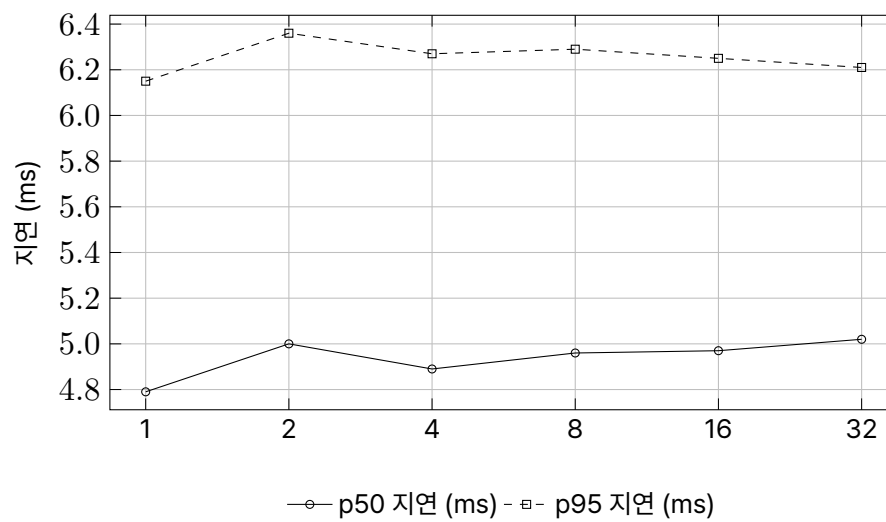


그림 2: 샤드 수

| 샤드 수 K | 이 샤드의 해시 키 | 이 샤드의 posting | RSS (MB) |
|--------|------------|---------------|----------|
| 1      | 2,320,053  | 5,166,499     | 1,001.7  |
| 2      | 1,159,639  | 2,584,077     | 616.6    |
| 4      | 580,088    | 1,290,508     | 400.2    |
| 8      | 289,835    | 644,698       | 292.2    |
| 16     | 145,011    | 322,356       | 237.6    |
| 32     | 72,302     | 160,996       | 205.7    |

posting 수는 K에 거의 정확히 반비례한다( $K=32$ 에서  $160,996 \approx 5,166,499/32$ ). 그런데 RSS는 그렇지 않다 -  $K=32$ 의 RSS(205.7MB)는  $K=1$ 의  $1/32$ (약 31.3MB)이 아니라 그보다 6.6배 크다. 두 점( $K=1, K=32$ )으로 “고정 오버헤드 + posting 당 비용” 모델을 적합해보면 고정 오버헤드는 약 180MB(인터프리터, numpy/scipy 로드, 트랙 하나를 처리하는 동안의 임시 버퍼), posting당 비용은 약 167바이트로 추정된다. 이 모델로 중간 지점( $K=4, 8, 16$ )을 역산하면 실측값과  $4^{15MB/2}$ (4%) 안에서 맞아 들어간다 - 정밀한 통계적 회귀는 아니지만, “샤드가 작아질수록 고정 오버헤드의 비중이 커진다”는 그림을 뒷받침한다. 샤드 용량을 아주 작게 잡으면 오히려 오버헤드가 지배적이 되어 샤딩의 이득이 줄어드는 지점이 있다는 뜻이다 - 4.3절의 플릿 총량 계산에 그대로 나타난다.

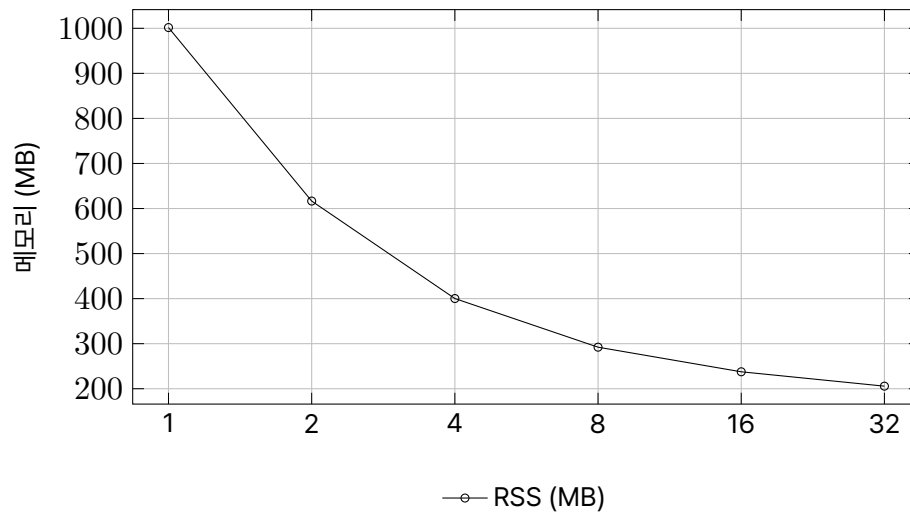


그림 3: 고정 카탈로그

### 5.3 100만곡 설계 - 합성 fill 실측 + 산술 투영(명시)

| 샤드 용량 C(트랙) | 이 용량의 샤드 메모리 (실측) | 100만곡에 필요한 샤드 수 $K=\text{ceil}(1M/C)$ | 플릿 전체 메모리 (산술) | 모놀리식 외삽 (868GB) 대비 |
|-------------|-------------------|---------------------------------------|----------------|--------------------|
| 1,000       | 1.89GB (실측)       | 1,000                                 | 1,885GB (산술)   | 2.17배              |
| 3,000       | 3.71GB (실측)       | 334                                   | 1,240GB (산술)   | 1.43배              |
| 6,000       | 6.13GB (실측)       | 167                                   | 1,024GB (산술)   | 1.18배              |

“샤드 메모리”는 3절에서 화이트노이즈 합성 fill로 실제로 그만큼의 트랙을 담아 측정한 값이다 - 100만곡 카탈로그에서도 샤드 하나가 담당하는 트랙 수는 여전히 C개뿐이므로, 이 숫자는 다시 외삽할 필요가 없다. 반면 “필요한 샤드 수”와 “플릿 전체 메모리”는 그 실측값에 단순 곱셈을 적용한 산술이다(직접 100만곡을 실행해 측정한 값이 아니다). 아래 차트는 이 설계를 모놀리식 외삽과 나란히 보여준다 - 모놀리식 곡선은 카탈로그가 커질수록 폭발하고, 샤딩 곡선은 어떤 C를 고르든 평평하다(그 C값에서 고정된 채로 유지된다). 정확하게 짚어야 할 것은 이 표의 오른쪽 열이다 - 샤딩은 플릿 전체 메모리를 줄이지 않는다. 오히려 샤드가 작을수록(용량 1,000트랙) 고정 오버헤드가 1,000번 중복돼 플릿 총량이 모놀리식 외삽치의 2.17배까지 불어난다. 용량을 키울수록(6,000트랙) 오버헤드 비중이 줄어 1.18배까지 좁혀지지만, 그래도 총량이 줄어드는 방향은 아니다.

샤딩이 실제로 바꾸는 것은 “한 노드가 감당해야 하는 상한”이다 - 868GB 머신 한 대는 상용 클라우드에 없지만, 6.13GB 서버 167대는 오늘 당장 구성 가능하다.

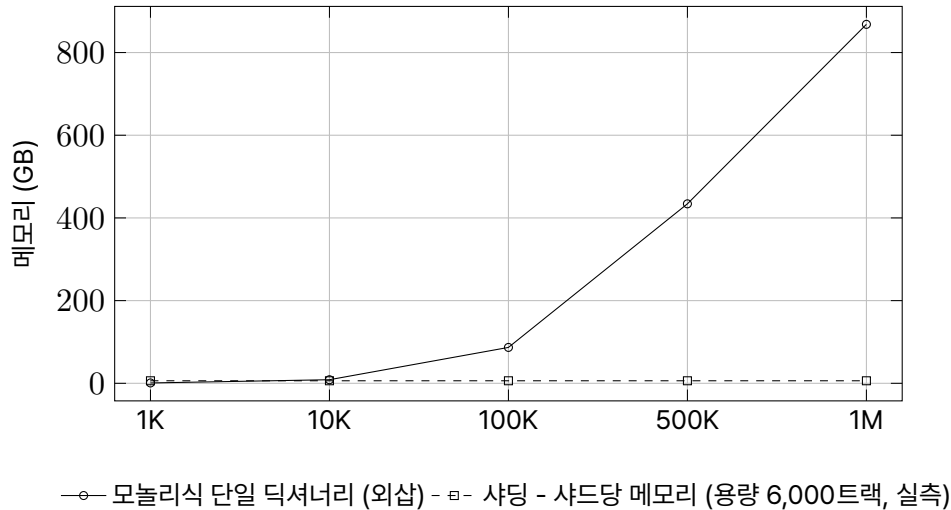


그림 4: 카탈로그 크기별 메모리 - 모놀리식 외삽 vs 샤딩 실측

## 6 한계

가장 먼저 짚을 것 - 이 백서의 샤딩은 **정확한 파티션**이고, 제목이 암시하는 “근사(approximate)” 인덱스(예: LSH 밴딩으로 쿼리를 일부 샤드에만 라우팅해 접촉 샤드 수 자체를 줄이는 설계)는 설계로만 논의했지 측정하지 않았다. 4.1절이 보여준 “접촉 샤드 비율  $\approx K$ ”가 정확한 파티션의 진짜 대가이고, 근사 라우팅은 이 대가를 줄일 수 있지만 recall을 다시 위험에 노출시킨다 - 그 트레이드오프는 이번 실험의 범위 밖이다.

둘째, 지연 측정은 한 프로세스 안에서 K개 디렉터리를 순차 조회한 시뮬레이션이다. 실제 분산 배치라면 샤드 대부분에 접촉해야 하는 이 설계에서 샤드당 네트워크 왕복이 더해지고, 이 백서는 그 비용을 측정하지 않았다 - 실제 지연은 이 표의 값보다 상당히 늘어날 수 있다.

셋째, 클린 5초 조건만 K별로 재측정했다. 라우팅이 해시 키의 순수 함수라는 설계 논증상 잡음·MP3·속도변화 조건에서도 recall이 동일해야 하지만, 이 백서는 그 논증을 클린 조건 하나로만 실측 확인했고 나머지 왜곡 조건에서 독립적으로 재검증하지는 않았다.

넷째, 실오디오 최대 규모는 로컬에서 확보 가능한 GTZAN 999트랙이고, 합성 file도 6,000트랙까지만 실측했다. 100만곡 설계(4.3절)는 이 두 앵커 사이/바깥을 잇는 산술 투영이며, 100만 트랙을 실제로 인덱싱해 측정한 결과가 아니다.

다섯째, 합성 file은 화이트노이즈다 - 이전 백서가 실측한 실제 음악(FMA)의 트랙당 약 0.889MB보다 posting 밀도가 몇 배 높게 나왔다(예: 합성 6,000트랙에서 트랙당 posting이 약 11,953개인 반면 GTZAN 실오디오는 트랙당 약 5,175개). 그래서 4.3절의 샤드 메모리 수치는 실제 음악 카탈로그가 필요로 할 값보다 보수적으로(더 크게) 잡혀 있을 가능성이 있다 - 방향은 유리한 쪽이지만 확정된 사실은 아니다.

여섯째, GTZAN은 원저작자가 명시한 정식 라이선스가 확인되지 않아 이번에도 로컬 알고리즘 검증에만 쓰고 재배포하지 않았다. 실제 상용 카탈로그는 라이선스 확보라는 기술과 무관한 별도 병목을 안고 있다.

## 7 재현

알고리즘: STFT(nperseg=2048, noverlap=1024), 15×15 국소 최댓값 피크픽킹 (노이즈 플러 = 평균 + 1.5표준편차, 트랙당 상위 1,500개 캡), 피크쌍 해싱 (팬아웃 8, 최대 시간차 40 타임빈). 샤딩 라우팅은  $(f1 \cdot a \text{ XOR } f2 \cdot b \text{ XOR } dt \cdot c) \bmod K$  형태의 고정 승수 해시 - 해시 키의 순수 함수이며 트랙 신원과 무관하다.

측정: (1) 실오디오 recall/지연은 GTZAN 999트랙, 150개 쿼리(고정 시드), 클립 5초 클린 조건,  $K = 1/2/4/8/16/32$ . (2) 단일 샤드 메모리는 998트랙 카탈로그를 스트리밍으로 처리하며 목표 샤드에 속하지 않는 해시는 즉시 버리는 격리 서브프

로세스로 측정(`resource.getrusage().ru_maxrss`). (3) 합성 fill은 화이트노이즈(`np.random`)를 동일한 파이프라인에 통과시켜 1,000/3,000/6,000 트랙 규모에서 같은 방식으로 모놀리식 메모리를 측정했다. 전부 CPython 3.12.8, CPU 전용, 로컬 단일 머신(48GB RAM/12코어)에서 실행했으며 GPU나 외부 API는 쓰지 않았다.

인용한 이전 실측(GTZAN 999트랙 알고리즘 검증, FMA-small 500→7,996트랙 스케일 검증, 트랙당 0.889MB/RSS 7,477MB, 868GB 외삽)은 이 백서와 같은 발행처의 별도 백서 “대규모 오디오 콘텐츠 지문 인식”(참고문헌 4)에서 가져왔다.

## 8 참고문헌

1. Wang, A. (2003). An Industrial-Strength Audio Search Algorithm. Proceedings of the 4th International Conference on Music Information Retrieval (ISMIR).
2. Tzanetakis, G., & Cook, P. (2002). Musical genre classification of audio signals. IEEE Transactions on Speech and Audio Processing, 10(5), 293-302. (GTZAN 데이터셋)
3. Karger, D., Lehman, E., Leighton, T., Panigrahy, R., Levine, M., & Lewin, D. (1997). Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web. Proceedings of the 29th Annual ACM Symposium on Theory of Computing (STOC).
4. 2i (2026). Large-Scale Audio Content Fingerprinting: Robust Song Identification Under Noise, Compression, and Tempo Shift, and the Honest Limits of Catalog Scaling. (2i 백서, FMA-small 카탈로그 스케일 실측)
5. Defferrard, M., Benzi, K., Vandergheynst, P., & Bresson, X. (2017). FMA: A Dataset for Music Analysis. Proceedings of the 18th International Society for Music Information Retrieval Conference (ISMIR).